REGEN AI PREP

INTERVIEW QUESTIONS

RAG & AI AGENTS

INTERVIEW QUESTIONS

with Answers to Crack

TECHNICAL INTERVIEW

Role:	ReGen -- AI Developer Intern (RAG / Agents)
Focus:	AI/ML fundamentals, LLMs, RAG, Agents, LangChain ecosystem, Python
Format:	55 questions across 11 topics, with humanised answers and code examples

Disclaimer

Everyone learns uniquely.

Master RAG, LLM and Agent concepts through these interview questions -- written the way you'd actually explain them out loud.

Take the help of this doc, and crack your ReGen technical round like a pro.

Table of Contents

01 AI, ML, Deep Learning & LLM Basics

02 Supervised, Unsupervised & Reinforcement Learning

03 Model Evaluation Metrics

04 Classic Machine Learning Algorithms

05 LLM & Transformer Internals

06 Retrieval-Augmented Generation (RAG)

07 Chunking, Embeddings & Vector Databases

08 AI Agents

09 LangChain, LangGraph & LangSmith

10 RAG Evaluation, Failures & Latency

11 Python & Backend for AI Systems

Topic 1

AI, ML, Deep Learning & LLM Basics

Q.1 What is the difference between AI, Machine Learning, and Deep Learning?

Ans: AI is the broad goal of building machines that behave intelligently. Machine Learning is a subset of AI where a system learns patterns from data instead of following hard-coded rules. Deep Learning is a subset of ML that uses multi-layer neural networks, which can learn complex patterns directly from raw data without manual feature engineering.

Q.2 What is a Large Language Model (LLM)?

Ans: An LLM is a deep learning model, built on the Transformer architecture, trained on massive amounts of text to predict the next token in a sequence. Because it has seen so much text during training, it becomes capable of writing, summarizing, translating, and answering questions in natural language.

Q.3 What is Generative AI, and how is it different from traditional ML?

Ans: Traditional ML mostly predicts a label or number from input data -- for example, spam or not spam. Generative AI creates new content -- text, images, audio, or code -- rather than just classifying something. LLMs are the text-focused branch of Generative AI.

Q.4 What is a token, and why do LLMs use tokens instead of whole words?

Ans: A token is the basic unit of text an LLM processes -- often a whole word, but sometimes just a part of one. Breaking text into sub-word tokens lets the model handle rare words, typos, and new words it has never seen, by combining smaller known pieces.

```
text = "unbelievable"
tokens = ["un", "believ", "able"]
print(len(tokens)) # Output: 3
```

Q.5 What is prompt engineering?

Ans: Prompt engineering is the practice of carefully wording the input given to an LLM -- with clear instructions, examples, or a required output format -- so it reliably produces the response you actually want, instead of a vague or inconsistent one.

Topic 2

Supervised, Unsupervised & Reinforcement Learning

Q.1 What is supervised learning?

Ans: Supervised learning trains a model on labeled data -- each input comes with a known correct output. The model learns the mapping between them so it can predict the output for new, unseen inputs.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression()
model.fit(X_train, y_train) # X_train, y_train are labeled
preds = model.predict(X_test)
```

Q.2 What is unsupervised learning?

Ans: Unsupervised learning works on data with no labels at all. Instead of predicting a known answer, the model finds hidden structure in the data on its own -- for example, grouping similar customers together using clustering.

Q.3 What is the difference between classification and regression?

Ans: Both are supervised learning tasks, but they predict different kinds of output. Classification predicts a category, such as spam or not spam. Regression predicts a continuous number, such as the price of a house.

Q.4 What is reinforcement learning?

Ans: Reinforcement learning is a different paradigm altogether -- an agent takes actions in an environment and receives rewards or penalties. Over many attempts, it learns a strategy that maximises the total reward, without ever being told the 'correct' action directly.

Q.5 What is semi-supervised learning, and when would you use it?

Ans: Semi-supervised learning combines a small amount of labeled data with a much larger amount of unlabeled data. It's useful when labeling data is expensive or slow, but you still have plenty of raw, unlabeled data available to help the model generalise better.

Topic 3

Model Evaluation Metrics

Q.1 What is a confusion matrix?

Ans: A confusion matrix is a table that breaks predictions into four buckets: True Positive, True Negative, False Positive, and False Negative. Every other classification metric -- accuracy, precision, recall, F1 -- is calculated directly from these four numbers.

Q.2 What are precision and recall, and how are they different?

Ans: Precision answers: of everything the model predicted as positive, how much was actually positive? Recall answers: of everything that was actually positive, how much did the model actually find? A model can be high on one and low on the other, which is why both matter together.

Q.3 Why would you use F1 score instead of plain accuracy?

Ans: Accuracy can be misleading on imbalanced datasets -- if 95% of emails are 'not spam', a model that always predicts 'not spam' still scores 95% accuracy while catching zero actual spam. F1 score balances precision and recall together, so it doesn't let a model hide behind one strong number.

Q.4 What is cross-validation, and why not just use a single train/test split?

Ans: A single train/test split gives you one performance number, which could be a lucky or unlucky split. Cross-validation splits the data into several folds, trains and tests multiple times across different folds, and averages the results for a far more reliable estimate.

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(model, X, y, cv=5, scoring='f1_macro')
print(scores.mean())
```

Q.5 What is overfitting versus underfitting?

Ans: Overfitting means the model memorised the training data too closely and performs poorly on new data -- high training accuracy, low test accuracy. Underfitting means the model is too simple to capture the real pattern, so it performs poorly on both training and test data.

Topic 4

Classic Machine Learning Algorithms

Q.1 How does linear regression work?

Ans: Linear regression fits a straight line through the data that best predicts a continuous output from one or more input features, by minimising the squared difference between predicted and actual values.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
print(model.predict(X_test))
```

Q.2 How does logistic regression work, and why is it used for classification?

Ans: Despite the name, logistic regression is used for classification. It takes a linear combination of the inputs and passes it through a sigmoid function, which squashes the output into a probability between 0 and 1 -- above a threshold, it's classified as one class; below, the other.

Q.3 How does the K-Nearest Neighbours (KNN) algorithm work?

Ans: KNN classifies a new data point by looking at the 'k' closest points to it in the training data and taking a majority vote of their labels. It's simple and effective, but sensitive to feature scale, so features are usually normalised first.

Q.4 How does a decision tree work, and how is a random forest different?

Ans: A decision tree repeatedly splits the data on the feature that best separates the classes, like a series of yes/no questions. A single tree can overfit easily. A random forest builds many trees on random subsets of the data and features, then combines their votes -- which is much more stable and harder to overfit.

Q.5 How does K-Means clustering work, and how do you choose k?

Ans: K-Means picks k random starting points as cluster centres, assigns every data point to its nearest centre, then moves each centre to the average of its assigned points -- repeating until the centres stop moving. The 'elbow method' -- plotting error against different values of k and looking for the bend in the curve -- is a common way to choose k.

Topic 5

LLM & Transformer Internals

Q.1 What is an embedding?

Ans: An embedding is a numerical vector that represents the meaning of a piece of text, positioned in space so that similar meanings sit close together. It's what makes semantic search and RAG possible, since you can compare meaning instead of just matching exact words.

```python
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')
vector = model.encode("machine learning is fun")
print(vector.shape) # e.g. (384,)
```

Q.2 What is a context window?

Ans: The context window is the maximum amount of text, measured in tokens, that a model can consider at one time. Anything beyond that limit either gets truncated or has to be summarised and fed in separately.

Q.3 What is self-attention, and why is it important?

Ans: Self-attention lets every token in a sequence look at every other token and decide how relevant it is, regardless of distance. This is what lets Transformers understand long-range context far better than older models that only read left to right, one word at a time.

Q.4 What is the difference between an encoder and a decoder in a Transformer?

Ans: An encoder reads the entire input at once and builds a rich understanding of it -- good for tasks like search or classification. A decoder generates output one token at a time, using what it has generated so far -- which is what most chat-style LLMs like GPT and Llama mainly use.

Q.5 What is hallucination, and how can it be reduced?

Ans: Hallucination is when a model generates confident-sounding text that is factually wrong or unsupported by any real source. It can be reduced by grounding the model with retrieved context (RAG), lowering the temperature setting, and explicitly instructing the model to answer only from the given context and say 'I don't know' otherwise.

Topic 6

Retrieval-Augmented Generation (RAG)

Q.1 What is RAG, and why does it exist?

Ans: An LLM only knows what it saw during training, and can't fit an entire knowledge base into one prompt. RAG solves this by retrieving the most relevant piece of external information for a query first, then feeding it into the LLM's prompt as context -- so the answer stays grounded in real, current information.

Q.2 What are the main components of a RAG pipeline?

Ans: A RAG pipeline has two halves: an offline indexing stage and an online query stage.

- **Indexing:** load documents → split into chunks → embed each chunk → store in a vector database
- **Query time:** embed the user's question → search the vector database for the closest chunks → pass those chunks + the question into the LLM's prompt → return the generated answer

```
# Simplified RAG query flow
query_vector = embed(user_question)
top_chunks = vector_db.search(query_vector, top_k=3)
prompt = build_prompt(user_question, top_chunks)
answer = llm.generate(prompt)
```

Q.3 What is the difference between RAG and fine-tuning?

Ans: RAG is like handing the model a reference manual to consult at query time -- its underlying skills don't change, but it gets fresh, external facts. Fine-tuning is like sending the model to a training course -- its actual behaviour, tone, or task-specific skill permanently changes. Use RAG when knowledge changes often or needs to come from private documents; use fine-tuning when you need to change how the model behaves or writes.

Q.4 What is chunking, and how do you decide chunk size?

Ans: Chunking means splitting long documents into smaller pieces before embedding them, so retrieval can pull back just the relevant section instead of an entire document. There's no single ideal chunk size -- you choose it based on document structure and validate it experimentally, keeping chunks big enough to preserve context but small enough to avoid pulling in irrelevant text.

Q.5 What is chunk overlap, and why does it matter?

Ans: Chunk overlap means repeating a small slice of text at the boundary between two consecutive chunks. Without it, a sentence that spans the cut point between two chunks can lose its meaning, because half of it ends up in one chunk and half in the next.

Topic 7

Chunking, Embeddings & Vector Databases

Q.1 What is a vector database, and how is it different from a normal database?

Ans: A normal database is searched by exact matches -- an ID, a name, a keyword. A vector database stores embeddings and is searched by similarity of meaning, so a query about 'vacation days' can match a document about 'annual leave' even though they share no exact words.

Q.2 What is cosine similarity, and why is it used for retrieval?

Ans: Cosine similarity measures the angle between two vectors rather than their raw distance, which makes it a reliable way to compare how similar two pieces of text are in meaning, regardless of their length.

```
import numpy as np

def cosine_similarity(a, b):
return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

score = cosine_similarity(query_vector, chunk_vector)
```

Q.3 What is the difference between Qdrant, ChromaDB, and PGVector?

Ans: Qdrant is a dedicated, production-grade vector search engine built for fast similarity search and filtering at scale. ChromaDB is lightweight and great for prototypes or local RAG projects. PGVector adds vector search directly inside PostgreSQL, which is useful when you already run Postgres and don't want a separate database.

Q.4 What is top-k retrieval?

Ans: Top-k retrieval means fetching only the k most similar chunks to a query, instead of every chunk in the database. A smaller k keeps the prompt focused; a larger k gives the LLM more context but risks pulling in irrelevant information.

Q.5 What is hybrid search and reranking?

Ans: Hybrid search combines keyword search (exact term matching) with vector search (semantic matching), catching cases where pure semantic search misses an exact term like a product code. Reranking takes the initially retrieved chunks and re-scores them with a more accurate (but slower) model, pushing the truly best matches to the top before they reach the LLM.

Topic 8

AI Agents

Q.1 What is an AI agent, and how is it different from a normal LLM app?

Ans: A normal LLM app takes one prompt and returns one answer. An agent can decide which tool or action to take, use it, look at the result, and repeat that loop -- reasoning through multiple steps -- until it reaches a final answer, similar to a personal assistant handling a multi-step task rather than answering a single question.

Q.2 What is tool calling / function calling?

Ans: Tool calling lets an LLM invoke an external function -- a search API, a database query, a calculator -- as part of its reasoning, instead of only generating text. The model decides which tool to call and with what arguments, based on a schema you define.

```
tool_schema = {
"name": "get_weather",
"description": "Get current weather for a city",
"parameters": {"city": "string"}
}
# The LLM decides when to call get_weather(city="Hyderabad")
```

Q.3 What is the ReAct pattern (the 'agent loop')?

Ans: ReAct stands for Reason + Act. The agent alternates between reasoning about what to do next, taking an action (like calling a tool), and observing the result -- repeating this cycle until it has enough information to give a final answer.

Q.4 What is agent memory?

Ans: Agent memory is how an agent keeps track of information across multiple turns or steps -- for example, remembering earlier parts of a conversation, or the results of tools it already called, so it doesn't repeat work or lose context mid-task.

Q.5 What is Agentic RAG?

Ans: In basic RAG, the system always retrieves once and generates an answer. In Agentic RAG, the agent decides whether retrieval is even necessary, evaluates whether the retrieved documents are actually relevant, and can rewrite the query and retrieve again if the first attempt wasn't good enough -- making the retrieval step itself part of the reasoning loop.

Topic 9

LangChain, LangGraph & LangSmith

Q.1 What is LangChain?

Ans: LangChain is a framework that provides ready-made building blocks for LLM applications -- prompt templates, retrievers, tools, and simple agents -- so you don't have to wire everything to the model API by hand.

Q.2 What is a chain in LangChain?

Ans: A chain is a sequence of steps linked together, where the output of one step becomes the input to the next -- for example, taking a user's question, retrieving context, formatting a prompt, and passing it to the LLM, all as one connected pipeline.

```
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain

prompt = PromptTemplate.from_template("Answer using context: {context}\nQ:
{question}")
chain = LLMChain(llm=llm, prompt=prompt)
chain.run(context=retrieved_text, question=user_question)
```

Q.3 What is LangGraph, and what do state, nodes, and edges mean?

Ans: LangGraph is a framework for building multi-step, stateful agent workflows, structured as a graph rather than a straight line.

- **State:** the data carried through the workflow (question, retrieved documents, current answer)
- **Node:** a function that performs one step (retrieve, check, generate)
- **Edge:** the routing logic deciding which node runs next -- including looping back if a check fails

Q.4 How is LangGraph different from LangChain?

Ans: LangChain gives you the higher-level building blocks for LLM apps and simple agents. LangGraph is for explicitly orchestrating complex, multi-step workflows that need branching or loops -- for example, retrieve, check if the documents are good enough, and loop back to retrieve again if they aren't.

Q.5 What is LangSmith used for?

Ans: LangSmith is an observability and evaluation layer for LLM applications. It traces every step of a run -- what was retrieved, what prompt was sent, what the model returned -- so you can debug exactly where something went wrong, and it supports building evaluation datasets to measure quality over time.

Topic 10

RAG Evaluation, Failures & Latency

Q.1 How would you evaluate a RAG system?

Ans: Evaluate retrieval and generation separately, since they can fail independently. For retrieval, check whether the right documents actually came back, using metrics like Recall@K. For generation, check correctness, relevance, and whether the answer stays faithful to the retrieved context -- ideally against a small test set of realistic questions with expected answers.

Q.2 How do you tell a retrieval failure apart from a generation failure?

Ans: Inspect what was actually retrieved for the failing question. If the right chunk never came back, that's a retrieval problem -- look at chunking, embeddings, or top-k. If the right chunk was retrieved but the LLM still gave a wrong answer, that's a generation problem -- look at the prompt and how the context is formatted.

Q.3 What do Recall@K and Precision@K mean?

Ans: Recall@K measures: of all the truly relevant documents that exist, how many appeared in the top K results? Precision@K measures: of the K results returned, how many were actually relevant? Together they show whether your retriever is both finding the right things and avoiding noise.

Q.4 How would you reduce latency in a RAG or agent pipeline?

Ans: First measure where the time is actually going -- the LLM call is almost always the biggest cost, not retrieval. Then trim the prompt and context size, use a faster model where accuracy allows, cache repeated computations and embeddings, run independent steps in parallel, and stream the response so it feels faster even before it's fully generated.

Q.5 How would you handle a question that has no answer in the knowledge base?

Ans: The retriever will still return the 'closest' chunks even if none of them are actually relevant, so the prompt should explicitly instruct the model to say it doesn't know rather than guess. You can also add a relevance threshold on the similarity score, and only pass chunks through to the LLM if they clear it.

Topic 11

Python & Backend for AI Systems

Q.1 How would you set up a FastAPI endpoint for a RAG chatbot?

Ans: Define a POST endpoint that accepts a question, runs it through the retrieval and generation pipeline, and returns the answer as JSON. Pydantic models validate the request and response shape automatically.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Query(BaseModel):
question: str

@app.post("/ask")
def ask(query: Query):
answer = rag_pipeline(query.question)
return {"answer": answer}
```

Q.2 How do you handle asynchronous LLM calls in Python?

Ans: LLM API calls are I/O-bound -- most of the time is spent waiting on a network response, not using the CPU. Using async/await lets your server handle other requests while waiting, instead of blocking the whole process on one slow call.

```
import asyncio

async def get_answer(question: str):
response = await llm_client.chat.completions.create(
messages=[{"role": "user", "content": question}]
)
return response.choices[0].message.content
```

Q.3 What is Pydantic, and why does FastAPI rely on it?

Ans: Pydantic defines the expected shape of data using plain Python classes, and automatically validates incoming requests against that shape -- rejecting malformed input with a clear error before it ever reaches your business logic.

Q.4 How would you unit test a RAG pipeline, given that LLM output isn't fully deterministic?

Ans: Split the pipeline into testable pieces: unit test the chunking and retrieval logic directly, since those are deterministic; for the LLM step, test that the output has the right structure and contains expected keywords or citations, rather than asserting an exact string match.

Q.5 What's the difference between REST and WebSocket, and when would you choose WebSocket?

Ans: REST is like sending a letter -- one request, one response, then the connection closes. A WebSocket is like a phone call -- the connection stays open, and both sides can keep sending messages in real time. Choose WebSocket for things like a live voice assistant or streaming chat, where you need continuous back-and-forth rather than a single request-response.